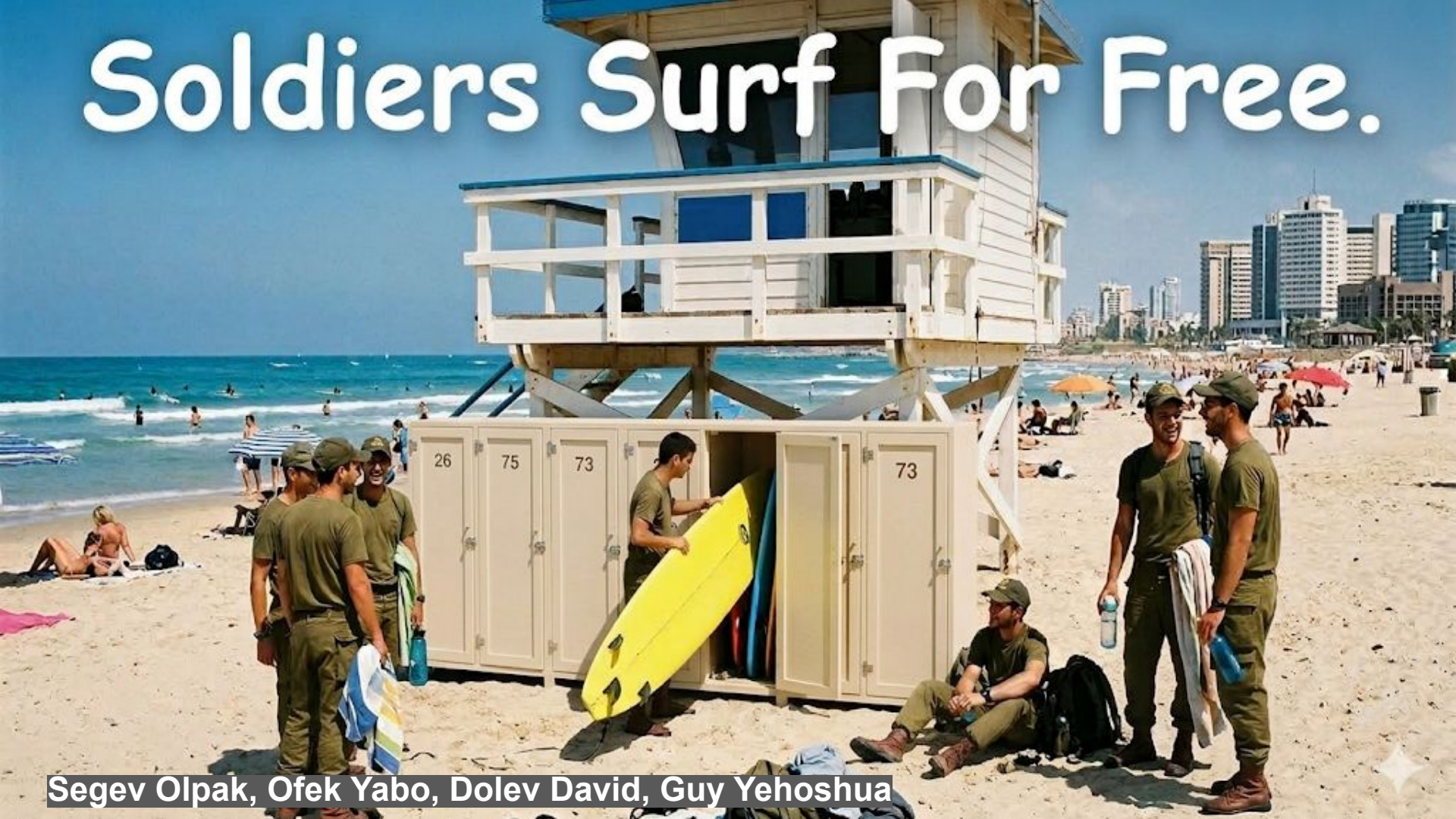

Soldiers Surf For Free.
26
75
73
73
Segev Olpak, Ofek Yabo, Dolev David, Guy Yehoshua

Landing Page & Poster

https://segevolp.github.io/Soldiers-Surf-Free/

Public Memorial & Marketing

Dedicated to Ido Aviv, telling the story behind the project.

Platform Showcase

Vision, hardware, team, architecture, and three user dashboards.

Centralized Documentation

Hosts all documents (SRD, TDD, final report, demo video) in one place.

Open Source Access

Direct links to the live system and the open-source GitHub repository.

Single-file responsive design with an ocean-inspired visual identity, hosted on GitHub Pages.

Borrow Flow Update

Reservation Mechanism
5-minute lock to prevent race conditions during concurrent borrow attempts.

Enhanced Security Guards
Strict validation ensures users successfully complete every mandatory step.

Revised PDF Behavior Workflow

1. Download Phase
- Fetch template
- Inject user credentials
- Upload to short-lived S3
- Send pre-signed URL

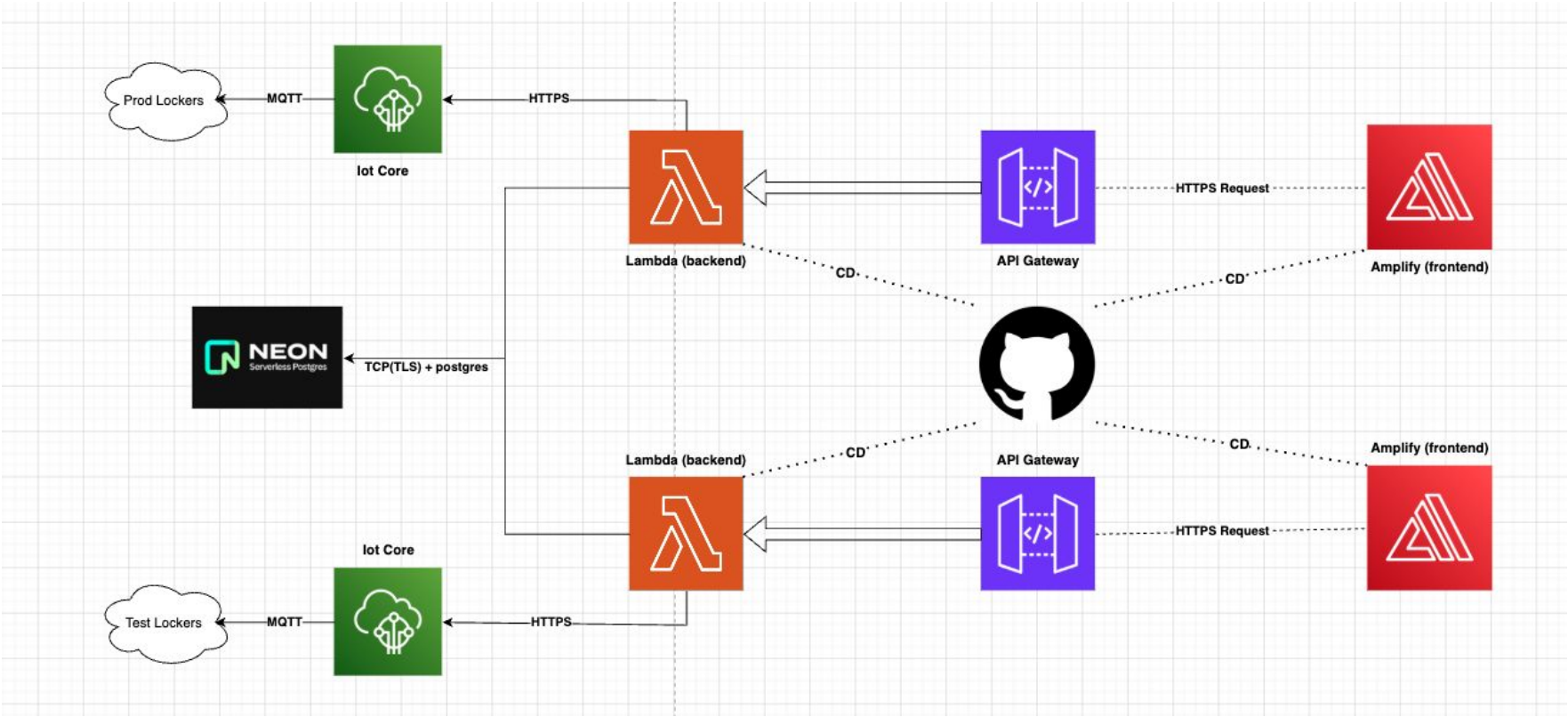
2. Accept Phase
- Record transaction in DB

3. Start Borrow Phase
- Verify acceptance status
- Download PDF from S3
- Email signed PDF to user

Performance Optimizations

- Added frontend route-level lazy loading for faster initial paint.
- Eliminated N+1 query behaviors to optimize backend processing.
- Streamlined API structure to minimize redundant DB I/O calls.
- Implemented DB Indexes for frequently queried data fields.
- Optimized asset delivery through frontend resource caching.

Development & Production Envs



Continuous Deployment (CD)

Dual-Pipeline Architecture

Automated split: Frontend managed by AWS Amplify and Backend via GitHub Actions.

Environment-Aware Routing

Smart branch mapping: 'development' deploys to Dev environment, 'main' to Production.

Backend Quality Gates

Pre-deployment validation: GitHub Actions spins up temporary PostgreSQL for full test suite execution.

Full Infra & DB Delivery

Beyond code: Automated Alembic DB migrations and infrastructure deployment via SAM tool.

Continuous Integration (CI)

Code Quality

flake8 + mypy on the backend. No database, fails fast on lint/type errors.

Frontend Build

Node 22, npm ci, then npm run build to confirm the React/Vite app compiles.

Automated Testing

Spins up Postgres 16, runs pytest with ≥ 80% coverage. Uploads results as artifacts.

Traceability

Reports even on failure. Fails build if test_index.json is stale to ensure sync.

Tests & Traceability

Structured Docstrings

Validated fields: TEST, LAYER, USE_CASE, INTENT, WHY_IT_MATTERS, FAILURE_MEANS, DEPENDENCIES, LAST_REVIEWED.

Full Traceability Chain

Test Use Case (UC-X.X) Requirement (REQ-XXX)

CI Traceability Job

tests.yml regenerates the index; fails build if test_index.json is stale or references unknown Use Cases.

Local Pre-commit Hook

Runs validation locally in ~2s. Catches sync issues before push without full test execution.